

IDE control room

Revise 360 lesson plan — 60 minutes

Unit	OCR J277 2.5 Programming languages and IDEs — Paper 2, Computational thinking, algorithms and programming
Lesson	IDE control room (lesson 4 of the unit)
Lesson ID	p1-l04
Experience ID	p1-l04
Plan ID	PLAN-PL-L04
Content version	14c83d06ff5b
Website	https://revise360.co.uk/experience.html?id=p1-l04

Big question, outcomes and success criteria

Big question — What help would you want from the software you use to write programs?

Prerequisites

The lesson before this one is p1-l03. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: p1-l01, p1-l02.

Assessable outcomes

1. **p1-l04-o1** — Identify the four main IDE facilities.
2. **p1-l04-o2** — Explain how each facility supports programming.
3. **p1-l04-o3** — Use editing, diagnostics and execution tools in an IDE.

Vocabulary, resources and preparation

Key vocabulary — IDE, Error diagnostics

Resources

SLIDES	presentations/PL_L04_IDEControlRoom_Lesson.pptx
WORKSHEET	worksheets/PL_L04_IDEControlRoom_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=p1-l04

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the 360 image is already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.

2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build APP_CONFIG.backendUrl is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on PL_L04_IDEControlRoom_Worksheet.docx is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong.	Correct their own starter.	A correction written, not just a tick.
8-18	Explain and model	The new idea in two or three steps, with the example worked in front of the class.	Listen and answer questions about the example.	They can say why, not only what.
18-21	Set up and settle	Sign-in, launch, and what to write before anyone taps an answer.	Sign in, open the lesson, find Tools in one place.	Everyone is in and on station 1.
21-43	Online: work the stations in order	Pupils read each wall, write the key fact and the challenge, then answer at the badge.	Read each wall, write the key fact and the challenge, then answer at the badge.	The worksheet has writing on it for each of the 6 stations before the badge is tapped.
43-48	Check understanding	The hinge question, hands down.	Answer the hinge question, hands down.	The split across the options.
48-56	Apply	Exam practice on the worksheet, in silence.	Exam practice, in silence.	worksheet, Exam practice a) to c)
56-60	Exit and set up the next lesson	The exit question and the retrieval link to next lesson.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Tools in one place

- An IDE combines programming facilities in one application.
- Editors and diagnostics help create and correct source.
- Translators and a run-time environment help execute it.

Challenge on this wall: How does an IDE help a programmer work on a program?

2. Writing source code

- Syntax highlighting uses colour to distinguish code elements.

- Auto-completion suggests names or code as you type.
- Indentation tools help present blocks clearly.

Challenge on this wall: How can auto-completion reduce mistakes without guaranteeing correct code?

3. The run-time environment

- A run-time environment lets a programmer execute a program.
- Input and output can be handled during the run.
- The programmer can observe the program's behaviour.

Challenge on this wall: What is the difference between the editor and the output area?

4. Diagnostic messages

- Diagnostics identify detected problems in the code.
- A message can include a line number and an error description.
- Read the surrounding code before deciding on a repair.

Challenge on this wall: A message points to line 5. Why might you also inspect line 4?

5. Translators in an IDE

- An IDE can use a compiler or an interpreter.
- The translator processes the program's source instructions.
- Translation and execution have different roles.

Challenge on this wall: Why does an IDE need access to a translator for high-level source?

6. Using your class IDE

- Open the IDE your teacher uses for programming.
- Use its editor, Run control and output area.
- Read a diagnostic message and correct its cause.

Challenge on this wall: In your class IDE, change a printed message and run it. Which two facilities did you use?

Retrieval and the check for understanding

Starter — A programmer needs to write source code and see its output. Name one tool for each job.

On the board or the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (p1 - 104-s4-c1), so the wording a pupil sees matches the wording you say.

Which repair addresses the visible syntax error on line 4?

Correct: Add the closing) after the string

What a pupil might choose instead

- Delete every print statement
- Change Ready to Go
- Change the font only

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: The opening bracket needs a matching closing bracket.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Recognise	14	The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.
Construct	1	Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Tools in one place	2	How does an IDE help a programmer work on a program?
2	Writing source code	2	How can auto-completion reduce mistakes without guaranteeing correct code?
3	The run-time environment	2	What is the difference between the editor and the output area?
4	Diagnostic messages	2	A message points to line 5. Why might you also inspect line 4?
5	Translators in an IDE	2	Why does an IDE need access to a translator for high-level source?
6	Using your class IDE	2	In your class IDE, change a printed message and run it. Which two facilities did you use?
final	Final challenge	3	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
p1-104-x1	Name two IDE editing facilities and explain how each helps a programmer.	4	worksheet, Exam practice a); deck, Exam practice slide
p1-104-x2	Explain how an error diagnostic helps a programmer correct a problem.	2	worksheet, Exam practice b); deck, Exam practice slide
p1-104-x3	Explain the roles of a translator and a run-time environment in an IDE.	4	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/p1-104.json`, which is not published with the website.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What is an IDE?** — A processor instruction set; A programming language itself
- **Does syntax highlighting prove that a calculation is correct?** — Yes, all coloured code is correct; Yes, if every line is blue
- **Which facility runs the program so you can observe its output?** — Editor colour theme; Auto-completion alone
- **Where would you look to see the result of `print(6 * 4)`?** — Only in the filename; Only in the colour legend
- **Which repair addresses the visible syntax error on line 4?** — Delete every print statement; Change Ready to Go

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Identify the four main IDE facilities.
- Explain how each facility supports programming.
- Use editing, diagnostics and execution tools in an IDE.

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is p1-105, Language review HQ. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.